



UCLL
HOGESCHOOL

Full-Stack software development

Lab 3: server components, Routing, hooks and Forms

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

Contents

1 INTRODUCTION	3
2 GETTING STARTED	4
3 DYNAMIC ROUTING	5
4 CLIENT COMPONENTS AND SERVER COMPONENTS	7
5 FORMS: BUILDING THE LOG-IN PAGE.....	8

1 Introduction

In this lab we will learn the basics of Next.js and how it uses React as a front-end library. We will create a Login form and see how we can adapt our lay-out when a user is logged in. We will only use front-end code.

NOTE: this looks like a big lab exercise, but you have 2 weeks to finish it.



The concepts covered include:

- Simple and dynamic app routing
- Server components and client components
- Forms: Binding values
- Forms: client-side validation
- Using the browser's storage
- Using the usePathname hook



In addition to the presentation slides, you can use the links below to successfully complete this lab:

- <https://react.dev/reference/react>
- <https://nextjs.org/docs>
- <https://nextjs.org/docs/app>

2 Getting started

To begin this lab, please accept the following GitHub assignment:

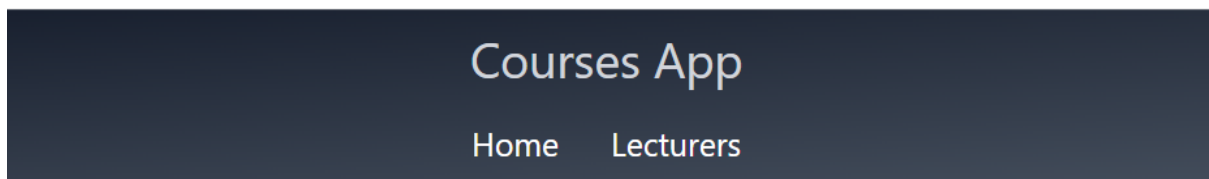
<https://classroom.github.com/a/qkKFtDi>

Once you have done this, you will be provided with a repository containing startcode. Clone it to a local directory and open it in VSCode.

Once you have accepted the assignment, a repository containing the start code will be created for you. Clone this repository to a local directory and open it in VSCode.

Follow the instructions in the README file.

In your browser, navigate to <http://localhost:8080>, which should look like this:



Welcome!

Courses lets you see as a lecturer all the courses you are teaching and as a student all the courses you are enrolled in. You can also see when the courses are scheduled and the students enrolled in each course.

3 Dynamic Routing

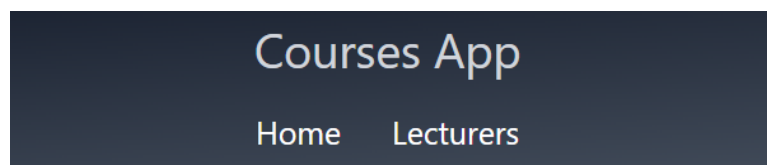
3.1 What you will learn



- ✓ How to create a dynamic route.
- ✓ How to read the data from the dynamic segments in a component

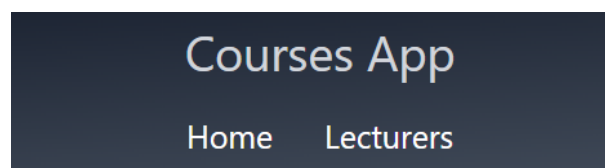
3.2 Tasks

With the application running, manually change the url to <http://localhost:8080/schedule/1/1>. You should see a 404 error (“this page could not be found”). Try to figure out why that is and make the changes to make this url work, so you get this result:



Hello from a dynamic route

As you can see, the 2 parameters aren’t used in the component. Change the `createSchedulePage` so you get this result when you go <http://localhost:8080/schedule/1/1>

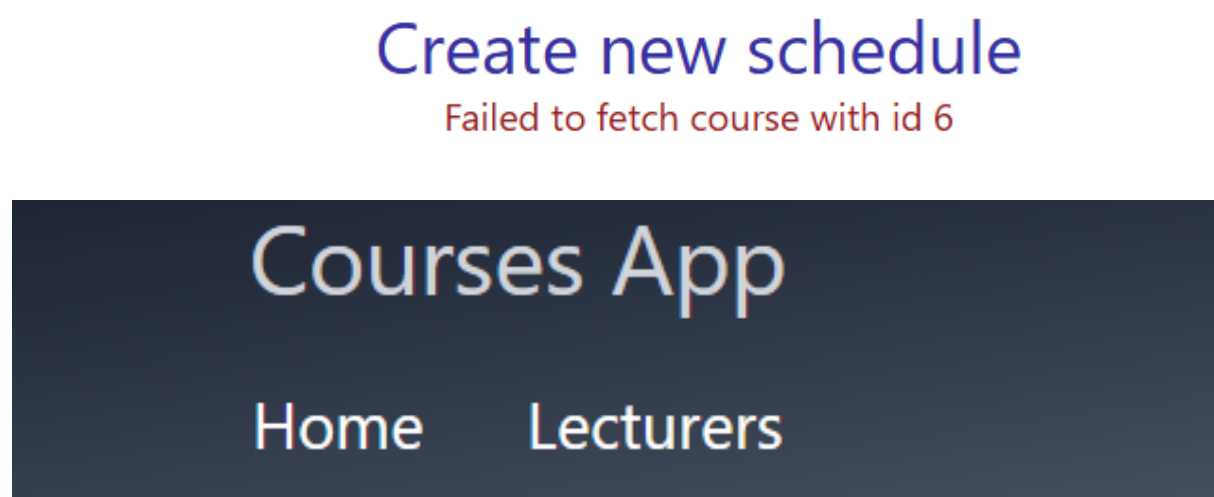
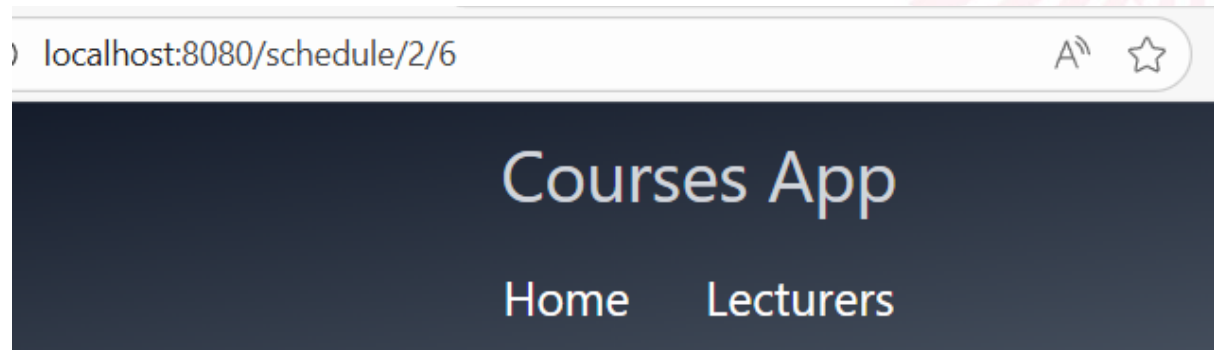


Create new schedule

Lecturer name: Johan Pieck
Course name: Full-stack development

Once we have a back-end in place, we will get back to this page. Try it with other ids in the the url too. (there are 4 courses and 4 lecturers, both with ids ranging from 1-4).

Now add error handling when one of the ids doesn't return a course or lecturer.



Create new schedule
Failed to fetch lecturer with id 6

4 Client Components and Server components

4.1 What you will learn



- ✓ How to create a server component.
- ✓ How to create a client component

4.2 Tasks

Look at the new Lecturers page. It is now a server component. However, this means that we can't use `useState` directly on this component.

Rebuild the page again, so it looks identical to the one in lab 2, but `Lecturerpage` must remain a server component.

Remember that components can only talk to their direct parents (with callbacks) and their direct children (with data props) How would you solve this problem?

(hint: create a new client component to keep the state)

5 Forms: building the log-in page

5.1 What you will learn



- ✓ How to create a form
- ✓ How to read and write from the browser's storage

5.2 Tasks

5.2.1 CREATE A LOGIN PAGE

In the app directory, add a folder named "login" and add this page.tsx

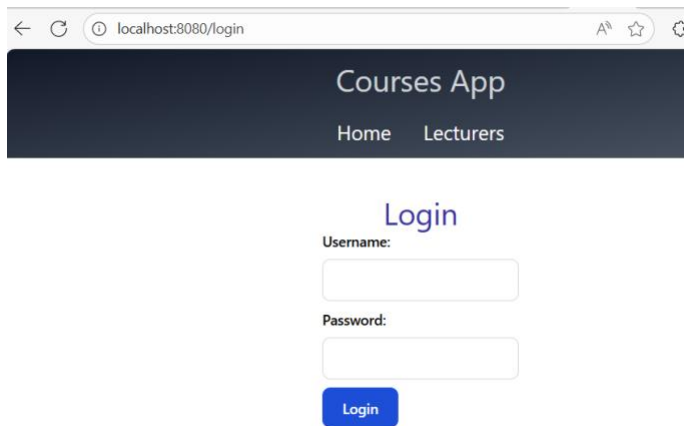
```
import UserLoginForm from "@components/users/UserLoginForm";

const LoginPage = () => {
  return (
    <>
      <h1 className="text-center">Login</h1>

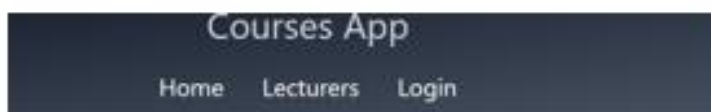
      <section className="flex flex-col justify-center">
        <UserLoginForm />
      </section>
    </>
  );
};

export default LoginPage;
```

This page uses a components named UserLoginForm that is located in the components/users folder. A very basic login form is already provided for you. If you manually enter the /login url, you should see this:



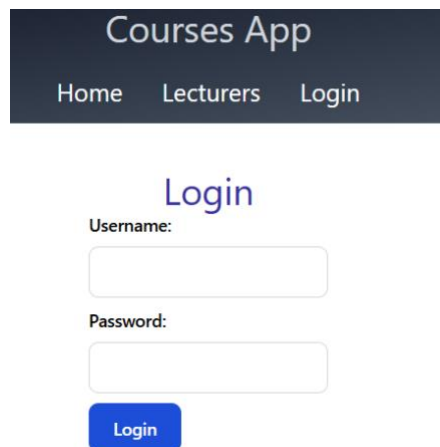
Now add a link to this page in the Header component.



5.2.2 BUILD THE LOGIN FORM

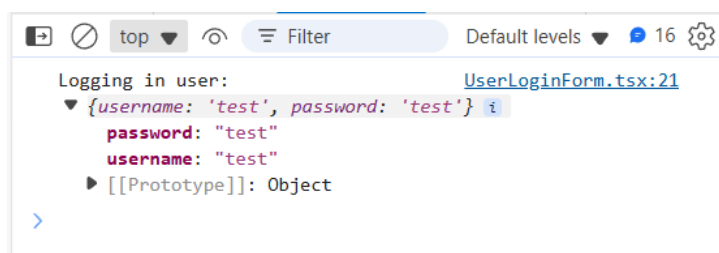
Handling form submit.

In the current form, nothing happens when we click the button.



As a test: connect the button click to a method that logs the user to the developer console.

This should look like this in your browser's devtools console:



5.2.3 ADDING FORM VALIDATION

It is best practice to validate forms in the front-end before submitting, whenever possible. This gives immediate feedback to the user before doing a round-trip to the back-end server.

First we need to create a state for `nameError` and `passwordError`, both of type `String | null`. After that, we need to add a section under each inputfield where we can put the validation messages.

The `handleSubmit` already calls the `validate` function, but it always returns `true`. Change it so it sets a `nameError` when the name is empty (“Username is required”) and/or sets a `passwordError` when the password is empty (“Password is required”). It should return `false` when either error is set.

When testing, you will notice that errors are persistent between clicks. We need to add a function to clear all errors and call it in the `handleSubmit` function.

If all this has been done correctly, pressing login when inputs are empty should look like this, and nothing should be logged to the console.

The image displays three sequential screenshots of a login form titled "Login".

- Left Screenshot:** The form is in a successful state. The "Username:" field contains the text "Jan" and the "Password:" field is empty. A red error message "Password is required" is visible below the password field. A blue "Login" button is at the bottom.
- Middle Screenshot:** The form shows a validation error for the username. The "Username:" field is empty, and a red error message "Username is required" is displayed below it. The "Password:" field is also empty, with a red error message "Password is required" below it. A blue "Login" button is at the bottom.
- Right Screenshot:** The form shows a validation error for the password. The "Username:" field is empty, with a red error message "Username is required" below it. The "Password:" field contains three dots "...", and a red error message "Password is required" is displayed below it. A blue "Login" button is at the bottom.

5.3 Mocking the login

Since we don't yet have a back-end, we will fake one in the `UserService`. This service will accept a username and password and will check if it exists in its hardcoded data. If the user exists he will be returned, otherwise an error is thrown.

In the `types/index.ts`, there is already a type defined for the `StatusMessage`. We can use that to tell the user whether the login attempt was successful or not. We will allow for multiple statusmessages to be shown at once by making the use state of type `StatusMessage[]`. Don't forget to clear the array in the `clearErrors` function.

We can show the statusmessages by using the `.map` function that returns a new array of the results of the callback function on every element of the original array:

```
return (  
  <div className="max-w-sm m-auto">  
    {statusMessages && (  
      <div className="row">  
        <ul className="list-none mb-3 mx-auto ">  
          {statusMessages.map(({ message, type }, index) => (  
            <li  
              key={index}  
              className={classNames({  
                'text-red-800': type === 'error',  
                'text-green-800': type === 'success',  
              })}>  
                {message}  
            </li>  
          ))}  
        </ul>  
      </div>  
    )}  
  </form onSubmit={handleSubmit}>
```

Try to guess what this does before testing it.

Now for the actual handling of the Service call:

```
try {
  const loggedInUser = await UserService.login(
    user.username,
    user.password
  );
  setStatusMessages([ { message: "Success", type: "success" } ]);
} catch (error) {
  setStatusMessages([ { message: (error as Error).message, type: "error" } ]);
}
};
```

Testing it out with the wrong name and password:

Courses App

HomeLecturersLogin

Login

Invalid username or password

Username:

Password:

Login

And with admin – admin:

Courses App

HomeLecturersLogin

Login

Success

Username:

Password:

Login

5.4 Storing the loggedInUser in browser storage:

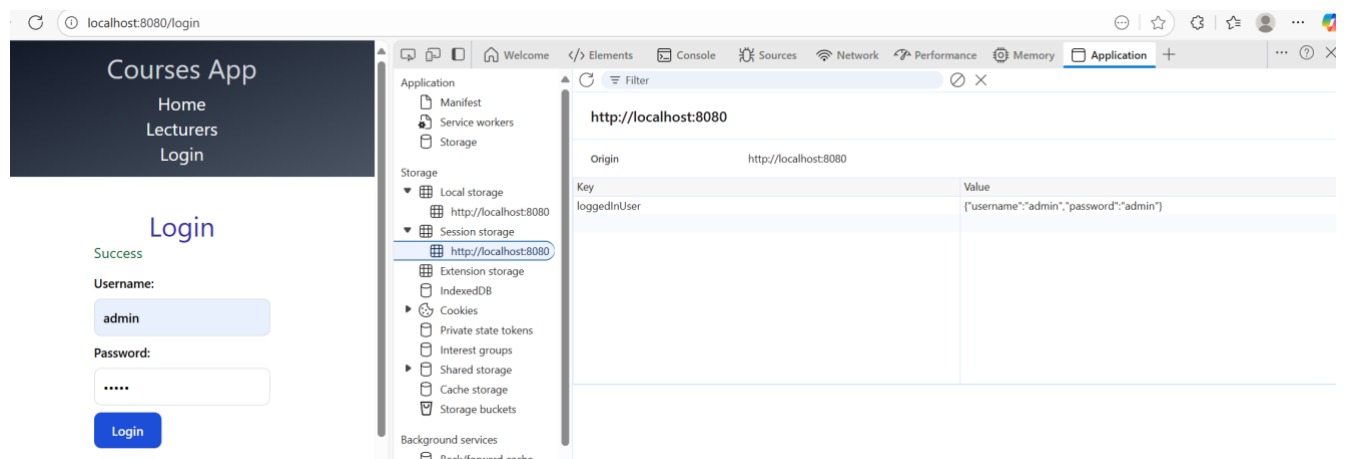
We have built a form, added validation and mocked a back-end call to check the data. But we haven't actually done anything with the logged in user. In this section, we are going to store the username of the user in the browser storage and retrieve it in another component.

In the UserLoginForm, put the user into the sessionStorage with the key "loggedInUser".



In a real-world application, we never store raw passwords in the session storage. This is very unsecure. We'll come back to this later.

To test it out, try to log in with admin – admin and open the devtools, you should see your key and values there.



5.5 Reading from the SessionStorage

Storing data in the SessionStorage means nothing if we can't read from it. That is what we're going to do now.

In your header.tsx, add the following:

- Use `useEffect` to read the logged-in user from session storage and set its username on the `loggedInUserName` state. This hook should run every time the url changes.

But wait... This causes an error.

You're importing a component that needs `useEffect`. This React Hook only works in a Client Component. To fix, mark the file (or its parent) with the `"use client"` directive.

By default, `header.tsx` is a server component. Server components can't use `useEffect`...

For the time being, we will change this component to a client component.

- Add 2 navigation items:
 - "Logout": only visible when a user is logged in
 - "hello, " followed by the logged-in user's name, only visible when a user is logged in
- Change the login navigation item to be only visible when there is no user logged in.
- Clicking on the logout button should erase the `loggedInUser` in session storage and then use the router to go to the `/` url.

In the `UserLoginForm`, we can make the process a bit more user-friendly:

- Change the text of the `successmessage` to:
"Login successful. Redirecting to homepage..."
- Use the `setTimeout()` function to use the router to go to the homepage after 1 second delay.
Test it out:

Login

Login successful. Redirecting to homepage...

Username:

admin

Password:

.....

Login



Welcome!

Courses lets you see as a lecturer all the courses you are teaching and as a student all the courses you are enrolled in. You can also see when the courses are scheduled and the students enrolled in each course.